MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (Honours)/ MGU-BBA (Honours)/ MGU-BCA (Honours)

REGULAR EXAMINATION October 2026

Fifth SEMESTER

**Discipline Specific Core Course (DSC) - MG5DSCECT302 - EMBEDDED SYSTEMS WITH AVR MICROCONTROLLER**

(2024 ADMISSION ONWARDS)

PART A

1. [b] Sets bit 2 of PORTB [K] / [CO1]

2. [a] TOV0 [U] / [CO2]

3. [b] Discrete angular steps Only one direction [U] / [CO3]

4. [b] EQU [K] / [CO1]

5. [d] 4 [K] / [CO2]

6. [a] Short-distance data exchange [U] / [CO3]

7. [d] 0AH [A] / [CO1]

8.[b] Store the value used for comparison with TCNT [U] / [CO2]

9. [c] Input [U] / [CO3]

10. [a] LDI R16, 25 [K] / [CO1]

PART B

11.Conditional Branching - Branching occurs only when a specified condition is satisfied. It depends on status flags such as Zero (Z), Carry (C), etc. Examples: BREQ, BRNE, BRCS. It is used for decision-making and loop control.
Unconditional Branching - Branching occurs without checking any condition. It does not depend on status flags. Examples: RJMP, JMP, IJMP. It is used to jump directly to another part of the program. [U] / [CO1]

12. In AVR microcontrollers, a time delay can be generated using a timer in Normal mode by loading an initial value into the timer counter and waiting for it to overflow.
*Select the timer:* Choose Timer0, Timer1, or Timer2 depending on the required delay.
*Configure Normal mode:* Set the timer control register bits (WGM) to 0 to select Normal mode.
*Set the pre-scaler:* Select a suitable pre-scaler value to divide the CPU clock and obtain the required timer frequency.
*Load the initial count:* Load the appropriate initial value into the timer counter register (TCNTn). The timer increments with each timer clock pulse.

*Wait for overflow:* When the timer reaches its maximum value and rolls over to zero, the overflow flag (TOVn) is set. Poll this flag or use an overflow interrupt. Clear the flag after detecting the overflow.
*Generate the delay:* Calculate the delay from the timer tick duration and the number of counts. Repeat the overflow process if a longer delay is required. [A] / [CO2]

13. Select a port pin: Choose a pin, for example, PB0 of the AVR microcontroller.
Configure as output: Set the corresponding bit in the Data Direction Register (DDRB) to 1.
Turn ON the LED: Set the corresponding bit in the Port Register (PORTB) to 1.
Turn OFF the LED: Clear the corresponding bit in PORTB to 0.
Connect the LED: Connect the LED to the output pin through a current-limiting resistor to prevent damage. An AVR port pin is configured as an output by setting its corresponding DDR bit to 1. The LED is switched ON or OFF by setting or clearing the corresponding PORT bit.
[An] / [CO3]

14. The main components of a microcontroller are: CPU (Central Processing Unit) / Memory / Input/Output (I/O) Ports / Timers/Counters / Communication Interfaces / Interrupt Controller and Clock Circuit.
A microprocessor is mainly designed for general-purpose computing, whereas a microcontroller integrates the CPU, memory and peripherals to control embedded applications.
*Microprocessor* - Mainly contains the CPU. Requires external memory and I/O interfaces. Generally used in computers and high-performance computing systems. Usually consumes more system power due to external components. Overall system cost is generally higher due to additional components.
*Microcontroller* - Contains CPU, memory and peripherals on a single chip. Usually has built-in memory and I/O peripherals. Generally used in embedded systems and control applications. Usually offers lower system power consumption. Overall system cost is generally lower for dedicated control tasks. [K] / [CO1]

15. c

```
#include <avr/io.h>

int main(void)
{
   DDRB = 0xFF;    // Configure Port B as output
   PORTB = 0xFF;   // Send Logic HIGH to all pins of Port B

   while (1)
   {
   }

   return 0;
}
```

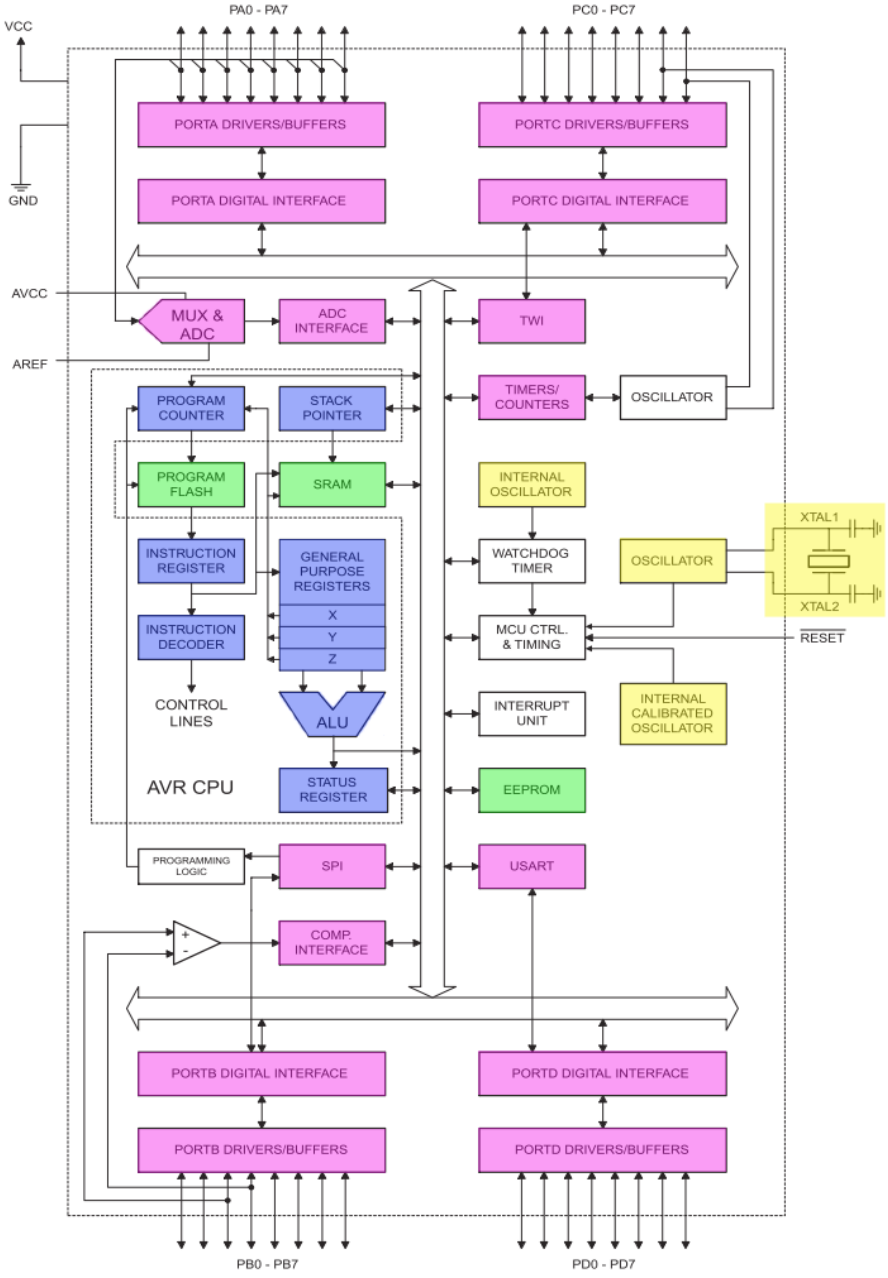
[A] / [CO2]

16. The Enable (EN) pin controls when the LCD accepts data or commands from the microcontroller. The microcontroller places the command or data on the LCD data pins before activating the EN pin. A pulse is applied to the EN pin to initiate the transfer of data or a

command. The LCD reads and latches the data or command, typically on the falling edge of the EN pulse in common HD44780-compatible LCDs. The EN pin synchronizes communication between the AVR microcontroller and the LCD. The Enable (EN) pin is used to control and synchronize the transfer of commands and data from the AVR microcontroller to the LCD. [U] / [CO3]

PART C

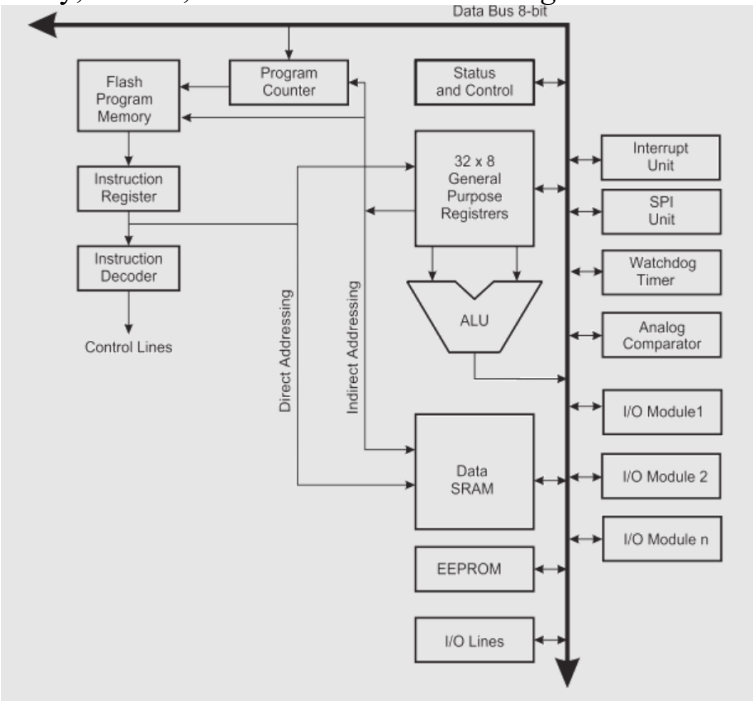
17.




*CPU (Central Processing Unit)* - 8-bit RISC processor / Executes instructions and controls all operations / Uses a Harvard architecture with separate program and data memory.
*Registers* - Contains 32 general-purpose 8-bit registers (R0–R31) / Used to store temporary data and operands / Three 16-bit pointer registers: X, Y and Z.

*ALU (Arithmetic Logic Unit)* - Performs arithmetic and logical operations / Supports addition, subtraction, AND, OR, XOR and comparison / Updates status flags in the Status Register (SREG).
*Program Memory* - 32 KB Flash memory / Stores the program instructions / Non-volatile memory.
*Data Memory* - 2 KB SRAM – stores temporary data and variables / 1 KB EEPROM – stores data permanently / Also includes I/O registers for controlling peripherals.
*Peripheral Interfaces* - I/O ports: 32 programmable I/O pins (Ports A, B, C and D) / Three timers/counters for timing and counting /10-bit, 8-channel analog-to-digital converter / USART, SPI and Two-Wire Interface (I²C-compatible) / Watchdog timer, interrupts and analog comparator. [U] / [CO1]

18. The ATmega32 is an 8-bit AVR RISC microcontroller that uses a Harvard architecture, in which program memory and data memory are separate. It has three main types of memory: Flash program memory, SRAM, and EEPROM. *Similar diagram can be used.*




The **Program Counter** is a 16-bit register that holds the address of the next instruction to be fetched from program memory. It automatically increments as instructions are executed. During jump, call, interrupt, or return operations, its value may change to the required address.
ATmega32 has 32 KB of Flash **program memory**. It stores the program instructions and constants. It is non-volatile, so data remains stored even when the power is switched off. It supports in-system programming and can be reprogrammed.
The **data memory** contains the register file, I/O registers, and internal SRAM. 32 general-purpose registers: R0–R31, each 8 bits wide. 64 I/O register locations: used to control peripherals and microcontroller functions. Extended I/O locations are also available in the ATmega32's data address space.
ATmega32 contains 2 KB of internal **SRAM.** It is volatile memory, meaning its contents are lost when power is removed. It stores variables, temporary results, function parameters, and local variables during program execution.
ATmega32 has 1 KB of **EEPROM**. It is non-volatile memory that retains data without power. It stores user settings, calibration values, and other data that must be preserved. It can be electrically erased and rewritten by the program.

The **stack** is a memory area located in SRAM and follows the LIFO (Last In, First Out) principle. The Stack Pointer (SP) is a 16-bit register that points to the top of the stack. The stack stores return addresses during function calls and interrupts, as well as temporary data and saved registers. [U] / [CO1]

19. The important registers and flags involved are: UDR (USART Data Register): Holds the character to be transmitted. UDRE (USART Data Register Empty): Indicates that the UDR can accept new data. TXC (Transmit Complete): Indicates that the entire character has been transmitted, including the stop bit. **Steps Involved in Transmitting a Character:**
Initialize USART: Set the baud rate using UBRRH and UBRRL, configure the frame format using UCSRC, and enable the transmitter using TXEN in UCSRB.
Check UDRE: Read the UDRE flag in UCSRA. If UDRE = 0, wait until the data register becomes empty.
Load the character: When UDRE = 1, write the character into UDR.
Transmit the character: The USART transfers the data to its shift register and sends it serially through TXD, including the start bit, data bits, optional parity bit, and stop bit.
Check TXC: Wait until TXC = 1 if the program needs to confirm that the complete character has left the transmitter.
**UDRE is a status flag** in the UCSRA register. It becomes 1 when the USART data register is ready to accept new data. It becomes 0 when new data is written to UDR, until the register becomes available again. It allows the program to load the next character without overwriting data that has not yet been transferred. It is useful for continuous or back-to-back character transmission.
**TXC is a status flag** in the UCSRA register. It is set to 1 when the entire frame in the transmit shift register has been shifted out and no new data is waiting in the transmit buffer. It indicates that transmission is physically complete, including the stop bit. It is useful when disabling the transmitter, switching a half-duplex communication line, or ensuring the last character has been sent before proceeding. TXC is cleared by hardware when its documented clearing conditions occur; on the ATmega32, it can also be cleared by writing a logic 1 to the TXC bit.
[K] / [CO2]

20. I²C (Inter-Integrated Circuit) is a synchronous serial communication protocol used to transfer data between a microcontroller and peripheral devices such as EEPROMs, sensors, RTCs, and LCD controllers. In AVR microcontrollers such as the ATmega32, I²C communication is implemented using the TWI (Two-Wire Serial Interface) module.I²C uses two signal lines: SDA (Serial Data Line): Transfers data between devices and SCL (Serial Clock Line): Carries the clock signal to synchronize data transfer.
The communication generally takes place as follows:

1. START condition: The master initiates communication by making SDA transition from HIGH to LOW while SCL is HIGH.
2. Address transmission: The master sends the 7-bit slave address followed by the read/write bit.
3. Acknowledgement (ACK): The addressed slave pulls SDA LOW during the acknowledgement clock pulse to indicate that it has received the address.
4. Data transmission: Data is transferred in groups of 8 bits, with the receiver acknowledging each byte.
5. Read or write operation: The R/W bit determines whether the master reads data from or writes data to the slave.
6. STOP condition: The master ends communication by making SDA transition from LOW to HIGH while SCL is HIGH.